OGC® DOCUMENT: 26-027

External identifier of this OGC® document: http://www.opengis.net/doc/profile/metocean-ogcapi-environmental-data-retrieval/1.0

Open Geospatial Consortium

METOCEAN PROFILE FOR OGC API - ENVIRONMENTAL DATA RETRIEVAL

STANDARD
Profile

DRAFT

Version: 1.0
Submission Date: 2026-05-08
Approval Date: 2026-05-08
Publication Date: 2026-05-08
Editor: Håvard Futsæter, Tom Kralidis

Notice for Drafts: This document is not an OGC Standard. This document is distributed for review and comment. This document is subject to change without notice and may not be referred to as an OGC Standard.
Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

License Agreement

Use of this document is subject to the license agreement at https://www.ogc.org/license

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: http://ogc.standardstracker.org/

Copyright notice

Copyright © 2026 Open Geospatial Consortium
To obtain additional rights of use, visit https://www.ogc.org/legal

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. KEYWORDS v

II. PREFACE vi

III. SECURITY CONSIDERATIONS vii

IV. SUBMITTING ORGANIZATIONS viii

1. SCOPE 2

2. CONFORMANCE 4

3. NORMATIVE REFERENCES 6

4. TERMS, DEFINITIONS AND ABBREVIATED TERMS 8
   4.1. Abbreviated terms 8

5. CONVENTIONS 11
   5.1. Identifiers 11
   5.2. Examples 11
   5.3. Schema representation 11
   5.4. Use of HTTPS 11

6. INTRODUCTION 14
   6.1. Project Aims & Goals 14
   6.2. Impact on users 15

7. CORE 17
   7.1. Requirements Class “Core” 17
   7.2. OpenAPI 17
   7.3. Collection identifier 18
   7.4. Collection title 19
   7.5. Collection license 19
   7.6. Collection spatial extent 20
   7.7. Collection temporal extent 20
   7.8. Collection vertical extent 21
   7.9. Collection parameter names 21
   7.10. Collection radius data query 22
   7.11. Locations query response format 23
   7.12. Error handling 23

7.13. Data query response metadata 24

8. INSITU OBSERVATIONS 26
8.1. Requirements Class “Insitu observations” 26
8.2. Collection data queries 26
8.3. Collection parameter names 27
8.4. Collection custom dimensions 27
8.5. CoverageJSON parameters 28

9. NUMERICAL WEATHER PREDICTION 31
9.1. Requirements Class “Numerical Weather Prediction” 31
9.2. Collection parameter names 31
9.3. Collection granularity 32
9.4. Collection data queries 32

10. DATA QUERY RESPONSE FORMAT 35
10.1. Requirements Class “Data query response format” 35
10.2. Encoding 35
10.3. CoverageJSON referencing 35

ANNEX A CONFORMANCE CLASS ABSTRACT TEST SUITE (NORMATIVE) 38

ANNEX B SCHEMAS (NORMATIVE) 48

ANNEX C EXAMPLES (INFORMATIVE) 50
C.1. Examples 50

ANNEX D NWP GLOBAL DETERMINISTIC PARAMETER NAME MATRIX 52

ANNEX E REVISION HISTORY 55

BIBLIOGRAPHY 57

I KEYWORDS

The following are keywords to be used by search engines and document catalogues.

metocean, ogc, api, edr, weather, meteorology, high value datasets

PREFACE

i. Abstract

The Open Geospatial Consortium (OGC) Environmental Data Retrieval (EDR) API provides a family of lightweight interfaces to access Environmental Data resources. Each resource addressed by an EDR API maps to a defined query pattern. OGC API — EDR identifies resources, captures compliance classes, and specifies requirements which are applicable to OGC Environmental Data Retrieval API's. The specification addresses two fundamental operations; discovery and query. Discovery operations allow the API to be interrogated to determine its capabilities and retrieve information (metadata) about this distribution of a resource. This includes the API definition of the server as well as metadata about the Environmental Data resources provided by the server. Query operations allow Environmental Data resources to be retrieved from the underlying data store based upon simple selection criteria, defined by this standard and selected by the client.

The flexibility of EDR allows for implementation in various environmental domains and communities of practice.

This document defines an EDR profile in support of providing access to meteorological datasets in the MetOcean community of practice. This includes, but is not limited to, conventions and constraints on identifiers, metadata parameters and encodings/formats and their related definitions and extensions.

ii. Keywords

The following are keywords to be used by search engines and document catalogues.

metocean, ogc, api, edr, weather, meteorology, high value datasets

iii. Security Considerations

No security considerations have been made for this document.

iv. Submitters

All questions regarding this document should be directed to the editor or the submitters:

Table — Submitters

NAME	AFFILIATION
Håvard Futsæter *(editor)*	MET Norway
Tom Kralidis *(editor)*	Meteorological Service of Canada



SECURITY CONSIDERATIONS

No security considerations have been made for this document.

IV SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- Norwegian Meteorological Institute
- Meteorological Service of Canada

1 SCOPE

1 SCOPE

This document defines the conventions, constraints and extensions of EDR in the context of the MetOcean community of practice.

The MetOcean EDR Profile defined herein is an extension of the International Standard *OGC API — Environmental Data Retrieval — Part 1: Core*.

This specification defines the conformance requirements for the MetOcean EDR Profile. Annex A defines the abstract test suite. Annex B provides normative information on schemas. Annex C provides informative examples.

2 CONFORMANCE

2 CONFORMANCE

Conformance with this standard shall be checked using the tests specified in Annex A (normative) of this document.

OGC API — Environmental Data Retrieval (EDR) provides a family of lightweight interfaces to access Environmental Data resources. Each resource addressed by an EDR API maps to a defined query pattern. This standard is a profile/extension of EDR. Conformance to this standard requires demonstrated conformance to the applicable Conformance Classes of *OGC API — Environmental Data Retrieval — Part 1: Core*.

Implementors of EDR API are required to comply with the MetOcean EDR Profile. A MetOcean EDR Profile shall therefore be compliant with OGC API — Environmental Data Retrieval — Part 1: Core.

This standard identifies the following Requirements Classes which define the functional requirements.

- “Core”
- “Observations”
- “Numerical Weather Prediction”

Compliant implementation of this profile requires conformance to at least the “Core” Requirements Class.

3 NORMATIVE REFERENCES

3 NORMATIVE REFERENCES

There are no normative references in this document.

There are no normative references in this document.

4

TERMS, DEFINITIONS AND ABBREVIATED TERMS

4 TERMS, DEFINITIONS AND ABBREVIATED TERMS

No terms and definitions are listed in this document.

No terms and definitions are listed in this document.

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “SHALL” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this Standard and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

The following additional terms and definitions also apply.

4.1. Abbreviated terms

Table 1 — Symbols and abbreviated terms

ABBREVIATION	TERM
API	Application Programming Interface
EU	European Union
EUMETNET	European National Meteorological and Hydrological Services
FAIR	Findable, Accessible, Interoperable, Reusable
FEMDI	Federated European Meteo-hydrological Data Infrastructure
GIS	Geographic Information System
HVD	High Value Datasets
HTML	Hypertext Markup Language

ABBREVIATION	TERM
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IANA	Internet Assigned Numbers Authority
IETF	Internet Engineering Task Force
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
MIME	Multipurpose Internet Mail Extensions
NMHS	National Meteorological and Hydrological Service
NWP	Numerical Weather Prediction
OGC	Open Geospatial Consortium
REST	Representational State Transfer
ROA	Resource-oriented architecture
RODEO	The provision of open access to public meteorological data and development of shared federated data infrastructure for the development of information products and services
S3	Simple Storage Service
SEO	Search engine optimization
SOA	Service-oriented architecture
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
W3C	World Wide Web Consortium
XML	eXtensible Markup Language

5 CONVENTIONS

5 CONVENTIONS

This section provides details and examples for any conventions used in the document. Examples of conventions are symbols, abbreviations, use of JSON and / or JSON schema, or special notes regarding how to read the document.

5.1. Identifiers

The normative provisions in this Standard are denoted by the URI:

`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0`

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.2. Examples

Examples provided in this specification are encoded as JSON, GeoJSON or CoverageJSON.

5.3. Schema representation

JSON Schema[1] objects are used throughout this standard to define the structure of metadata records. These schema objects are also typically represented using YAML[2]. YAML is a superset of JSON, and in this standard are regarded as equivalent.

[1] https://json-schema.org

[2] https://en.wikipedia.org/wiki/YAML

5.4. Use of HTTPS

For simplicity, this document only refers to the HTTP protocol. This is not meant to exclude the use of HTTPS and simply is a shorthand notation for “HTTP or HTTPS.” In fact, most servers are expected to use HTTPS, not HTTP.

6 INTRODUCTION

6 INTRODUCTION

RODEO, the Provision of Open Access to Public Meteorological Data and Development of Shared Federated Data Infrastructure for the Development of Information Products and Services, is a joint effort by eleven European National Meteorological and Hydrological Services (NMHS), the European Centre for Medium-Range Weather Forecasts (ECMWF) and the network of 31 European National Meteorological and Hydrological Services (EUMETNET). The Project Partners are listed in the page Partners[3].

The RODEO project develops a user interface and Application Programming Interfaces (API) for accessing meteorological datasets declared as High Value Datasets (HVD) by the EU Implementing Regulation (EU) 2023/138 under the EU Open Data Directive (EU) 2019/1024. The project also fosters the engagement between data providers and data users for enhancing the understanding of technical solutions being available for sharing and accessing the HVD datasets.

Surface weather observations, climate data, weather warnings, radar data and numerical weather prediction (NWP) data are defined as meteorological high value datasets. The distribution of these datasets is going to be made available under an open licence, in machine-readable formats using APIs and bulk downloads following the FAIR principles (usability, accessibility, interoperability and reusability).

The project is funded by the EU Digital Europe Program (DIGITAL) and EUMETNET.

6.1. Project Aims & Goals

The project makes meteorological High Value Datasets easily available with an aim to bring new data to businesses, public administrations and citizens. It reinforces the European public meteorological data infrastructure and enhances the providers' digital capacity to share data. Furthermore, the project also fosters the engagement between data providers and data users for enhancing the understanding of technical solutions to be available for sharing and accessing the datasets.

The project strengthens the capacity of the European meteorological data providers to comply with the HVD Implementing Regulation by:

- Developing a user interface
- Developing APIs for accessing weather observation data, climate data, weather radar data, warnings, and Artificial Intelligence (AI) datasets

[3] https://rodeo-project.eu/partners

- Developing a data catalogue for making data discoverable
- Engaging with the data owners and user communities
- Supporting the deployment of national data portals and APIs
- Making HVDs available from the project partners

6.2. Impact on users

The increased availability of data boosts entrepreneurship and potentially results in the creation of new companies. New open datasets are an important resource for small and medium-sized enterprises to develop new digital products and services. Reuse of data opens business opportunities for several sectors ultimately leads attracting more investors. By making data much easier to use, reseach, academia, AI applications and many other application areas can utilize the data more efficiently.

Overall, better data availability leads to better warnings, forecasts, and services to the public and weather-critical industries, and contributes to the safe and efficient functioning of society with multiple benefits across the European economy, industry, and society.

7 CORE

7 CORE

7.1. Requirements Class “Core”

REQUIREMENTS CLASS 1	
IDENTIFIER	`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
CONFORMANCE CLASS	Conformance class A.1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/core`
TARGET-TYPE	Web API
DEPENDENCY	OGC API — Environmental Data Retrieval Standard — Part 1: Core (2023)[4]
PRE-CONDITION	The API conforms to OGC API — Environmental Data Retrieval — Part 1: Core, Requirements Class: Core and Requirements Class: Collections
NORMATIVE STATEMENTS	Recommendation 1: `/rec/core/collection_vertical_extent` Recommendation 2: `/rec/core/data_query_response_metadata` Requirement 1: `/req/core/openapi` Requirement 2: `/req/core/collection_identifier` Requirement 3: `/req/core/collection_title` Requirement 4: `/req/core/collection_license` Requirement 5: `/req/core/collection_spatial_extent` Requirement 6: `/req/core/collection_temporal_extent` Requirement 7: `/req/core/collection_parameter_names` Requirement 8: `/req/core/collection_radius_data_query` Requirement 9: `/req/core/locations_query_response_format` Requirement 10: `/req/core/error_handling`

[4] https://docs.ogc.org/is/19-086r6/19-086r6.html

7.2. OpenAPI

An OpenAPI specification provides a machine-readable description of the API interface.

REQUIREMENT 1	
IDENTIFIER	`/req/core/openapi`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	An API definition SHALL be described using an OpenAPI 3.1 document.
A	The API definition SHALL be described using an OpenAPI document (version 3.1 or higher).
B	The OpenAPI document SHALL be encoded as one of the OpenAPI JSON/YAML media types.
C	The landing page SHALL link the OpenAPI document with rel=service-desc.
D	The landing page SHALL link API documentation with rel=service-doc and type text/html.

7.3. Collection identifier

A collection identifier provides a mechanism to uniquely identify a given collection in an OGC API.

REQUIREMENT 2	
IDENTIFIER	`/req/core/collection_identifier`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	A collection identifier SHALL NOT convey structured metadata and SHOULD use the fixed value list.
A	A collection identifier SHALL NOT be used to convey structured metadata.
B	A collection identifier SHOULD use one of insitu-observations, climate_data, radar_observations, weather_warnings, weather_forecast.

REQUIREMENT 2	
C	The identifier string MAY include a postfix to the listed values if needed.

7.4. Collection title

A collection title provides a human-readable short description of the given collection.

REQUIREMENT 3	
IDENTIFIER	`/req/core/collection_title`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	A title SHALL be set for all collections and SHALL NOT exceed 50 characters.
A	A title SHALL be set for all collections.
B	A title SHALL NOT have more than 50 characters.
C	A title SHOULD be written in English.
D	A title SHOULD place the most important information first to guard against truncation.
E	A title SHOULD describe the collection understandably for non-experts (topic/domain and geographical area).

7.5. Collection license

A license describes the usage permissions for the data in a collection.

REQUIREMENT 4	
IDENTIFIER	`/req/core/collection_license`

REQUIREMENT 4	
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	A collection SHALL link to a license of its data with rel=license and type=text/html.
A	The links property in a collection SHALL contain a link to a license of its data.
B	The license link SHALL set the rel property to license.
C	The type property SHALL be set to text/html.
D	If the data is open data, the license SHALL be CC BY 4.0 with href https://creativecommons.org/licenses/by/4.0/.

7.6. Collection spatial extent

Spatial extent of a collection.

REQUIREMENT 5	
IDENTIFIER	`/req/core/collection_spatial_extent`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	The collection spatial extent and CRS list SHALL use OGC:CRS84.
A	The value of extent.spatial.crs SHALL be OGC:CRS84.
B	The value of crs SHALL include the element OGC:CRS84.
C	If crs_details is specified for a data_queries object, it SHALL include an object whose crs value is OGC:CRS84.
D	Additional CRSs SHOULD be presented as EPSG:<code> in crs and data_queries.*.crs_details.crs.

7.7. Collection temporal extent

Temporal extent of a collection.

REQUIREMENT 6	
IDENTIFIER	`/req/core/collection_temporal_extent`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	The value of extent.temporal.trs SHALL be Gregorian.
A	The value of extent.temporal.trs SHALL be Gregorian.

7.8. Collection vertical extent

Vertical extent of a collection.

RECOMMENDATION 1	
IDENTIFIER	`/rec/core/collection_vertical_extent`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	If extent.vertical is defined, extent.vertical.vrs SHOULD use one of the recommended vertical reference system values.
A	If extent.vertical is defined, the value of extent.vertical.vrs SHOULD be one of “Pressure level in hPa”, “Geopotential height in gpm”, “Geometrical altitude above mean sea level in meters”, “Height above ground in meters” or “Flight level”.

7.9. Collection parameter names

Parameter names is an object containing parameter objects with metadata about all parameters the collection contains.

REQUIREMENT 7	
IDENTIFIER	`/req/core/collection_parameter_names`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	Parameter objects SHALL specify label, description and unit using the profile conventions.
A	The keys in parameter_names and id in the parameter object SHALL NOT convey structured metadata.
B	A parameter object SHALL specify the fields label, description and unit.
C	label SHALL NOT have more than 50 characters.
D	label SHALL be written in English.
E	unit SHALL use QUDT; unit.symbol.type SHALL be https://qudt.org/vocab/unit/<unit> and unit.symbol.value the qudt symbol.
F	observedProperty SHALL use the CF-convention (observedProperty.id = https://vocab.nerc.ac.uk/standard_name/<name>) when available; otherwise use observedProperty.description.

7.10. Collection radius data query

Metadata about the radius data query in a collection.

REQUIREMENT 8	
IDENTIFIER	`/req/core/collection_radius_data_query`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	A radius data query within_units array SHALL contain m (SI unit of length).

REQUIREMENT 8	
A	The within_units array in a variables object for a radius data query SHALL contain m.
B	within_units MAY contain more values, each defined as a unit at https://qudt.org using the unit symbol.
C	A radius query with a limit parameter SHOULD return the closest items to the radius centre.

7.11. Locations query response format

Structure of the response document for a /locations query.

REQUIREMENT 9	
IDENTIFIER	`/req/core/locations_query_response_format`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	A /locations response SHALL be a GeoJSON FeatureCollection with id and properties.name per feature.
A	The /locations response SHALL be a GeoJSON document with a FeatureCollection object.
B	Each Feature SHALL have id of type string and properties.name of type string.
C	If not all collection parameters are provided, properties.parameter-name SHALL list a subset of the parameter_names keys.
D	Links with more information SHALL be included in properties.links as OGC API — Common link objects.
E	A more detailed description MAY be provided via properties.description.
F	All strings SHALL be in English.

7.12. Error handling

HTTP status codes and response documents for responding to errors.

REQUIREMENT 10	
IDENTIFIER	`/req/core/error_handling`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	Empty results SHALL return HTTP 204 and 4xx responses SHOULD comply with RFC 9457.
A	A query for data inside the extent that results in no data SHALL return HTTP 204 with no content.
B	A query partially within the extent SHALL respond as within the extent, returning only the in-extent portion.
C	Any 4xx response SHOULD comply with RFC 9457 Problem Details for HTTP APIs.

7.13. Data query response metadata

Metadata in the response to a data query.

RECOMMENDATION 2	
IDENTIFIER	`/rec/core/data_query_response_metadata`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
STATEMENT	The metadata in a data query response SHOULD mirror the terms and vocabularies used in the collection metadata.
A	The metadata in the response to a data query (e.g. crs, parameters and units) SHOULD mirror as closely as possible the terms and vocabularies used in the collection metadata.

8

INSITU OBSERVATIONS

8 INSITU OBSERVATIONS

8.1. Requirements Class “Insitu observations”

REQUIREMENTS CLASS 2	
IDENTIFIER	`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/insitu-observations`
CONFORMANCE CLASS	Conformance class A.2: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/insitu-observations`
TARGET-TYPE	Web API
DEPENDENCY	Core Data query response format
NORMATIVE STATEMENTS	Recommendation 3: `/rec/insitu-observations/collection_parameter_names` Requirement 11: `/req/insitu-observations/collection_data_queries` Requirement 12: `/req/insitu-observations/collection_parameter_names` Requirement 13: `/req/insitu-observations/collection_custom_dimensions` Requirement 14: `/req/insitu-observations/coveragejson_parameters`

8.2. Collection data queries

The types of data queries implemented in a collection.

REQUIREMENT 11	
IDENTIFIER	`/req/insitu-observations/collection_data_queries`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/insitu-observations`
STATEMENT	An insitu-observations collection SHALL support locations, area and radius.

REQUIREMENT 11	
A	A collection SHALL support locations, area and radius.

8.3. Collection parameter names

Parameter names is an object containing parameter objects with metadata about all parameters the collection contains.

REQUIREMENT 12	
IDENTIFIER	`/req/insitu-observations/collection_parameter_names`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/insitu-observations`
STATEMENT	Insitu parameters SHALL specify metocean:standard_name, metocean:level and measurementType.
A	Each parameter SHALL specify metocean:standard_name set to a value from the standard_name custom dimension.
B	Each parameter SHALL specify metocean:level set to a value from the level custom dimension.
C	Each parameter SHALL specify measurementType.

RECOMMENDATION 3	
IDENTIFIER	`/rec/insitu-observations/collection_parameter_names`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/insitu-observations`
STATEMENT	Each measurementType.method in parameter_names SHOULD follow the CF-convention cell methods.
A	Each measurementType.method object in parameter_names SHOULD follow the CF-convention cell methods and use a value from the CF cell-methods appendix.

8.4. Collection custom dimensions

The custom dimensions for which you can query the collection.

REQUIREMENT 13	
IDENTIFIER	`/req/insitu-observations/collection_custom_dimensions`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/insitu-observations`
STATEMENT	An insitu-observations collection SHALL define standard_name and level custom dimensions.
A	The collection SHALL have an extent.custom object with id standard_name.
B	Custom dimension standard_name SHALL list CF standard names in values.
C	Custom dimension standard_name SHALL specify reference https://vocab.nerc.ac.uk/standard_name/.
D	The collection SHALL have an extent.custom object with id level.
E	Custom dimension level SHALL specify reference “Height of measurement above ground level in meters”.

8.5. CoverageJSON parameters

The metadata about parameters in CoverageJSON.

REQUIREMENT 14	
IDENTIFIER	`/req/insitu-observations/coveragejson_parameters`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/insitu-observations`
STATEMENT	CoverageJSON parameters SHALL carry metocean:measurementType, metocean:standard_name and metocean:level.
A	Each object in parameters in a coverage SHALL have metocean:measurementType with the same properties as measurementType.

REQUIREMENT 14	
B	Each object in parameters SHALL have metocean:standard_name.
C	Each object in parameters SHALL have metocean:level.

9

NUMERICAL WEATHER PREDICTION

9 NUMERICAL WEATHER PREDICTION

9.1. Requirements Class “Numerical Weather Prediction”

REQUIREMENTS CLASS 3	
IDENTIFIER	`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/nwp`
CONFORMANCE CLASS	Conformance class A.3: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/nwp`
TARGET-TYPE	Web API
DEPENDENCY	Core Data query response format
NORMATIVE STATEMENTS	Requirement 15: `/req/nwp/collection_parameter_names` Requirement 16: `/req/nwp/collection_granularity` Requirement 17: `/req/nwp/collection_data_queries`

9.2. Collection parameter names

Parameter names is an object containing parameter objects with metadata about all parameters the collection contains.

REQUIREMENT 15	
IDENTIFIER	`/req/nwp/collection_parameter_names`
INCLUDED IN	Requirements class 3: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/nwp`
STATEMENT	NWP parameters SHALL use ECMWF short names per Annex D for id and description.
A	WIPPS Manual Global Deterministic NWP parameters SHALL be provided using ECMWF short names per Annex D.

REQUIREMENT 15	
B	The parameter name SHALL be provided in the description property.
C	The ECMWF short name SHALL be provided in the id property.

9.3. Collection granularity

The organization of NWP data advertised as collections and instances.

REQUIREMENT 16	
IDENTIFIER	`/req/nwp/collection_granularity`
INCLUDED IN	Requirements class 3: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/nwp`
STATEMENT	An NWP collection represents a model; instances represent model runs identified by RFC3339 datestamps.
A	A collection SHALL represent and describe a model.
B	A collection instance SHALL represent and describe a model run.
C	A collection instance identifier SHALL be represented by an RFC3339 datestamp.
D	A collection temporal instance or interval SHALL represent a timestep or timesteps of a model run.

9.4. Collection data queries

The types of data queries implemented in a collection.

REQUIREMENT 17	
IDENTIFIER	`/req/nwp/collection_data_queries`

REQUIREMENT 17	
INCLUDED IN	Requirements class 3: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/nwp`
STATEMENT	An NWP collection SHALL support position and cube; no instance means latest.
A	A collection SHALL support position and cube queries.
B	Data queries without an instance SHALL be interpreted as the latest instance of a collection.
C	A datetime parameter between two timestamps SHALL trigger an HTTP 400 Bad Request.

10

DATA QUERY RESPONSE FORMAT

10 DATA QUERY RESPONSE FORMAT

10.1. Requirements Class “Data query response format”

REQUIREMENTS CLASS 4	
IDENTIFIER	http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/data-query-response-format
CONFORMANCE CLASS	Conformance class A.4: http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/data-query-response-format
TARGET-TYPE	Web API
DEPENDENCY	Core CoverageJSON
NORMATIVE STATEMENTS	Requirement 18: /req/data-query-response-format/encoding Requirement 19: /req/data-query-response-format/referencing

10.2. Encoding

The response format for data queries.

REQUIREMENT 18	
IDENTIFIER	/req/data-query-response-format/encoding
INCLUDED IN	Requirements class 4: http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/data-query-response-format
STATEMENT	All data queries SHALL support CoverageJSON as a response format.
A	All data queries SHALL support CoverageJSON as a response format.

10.3. CoverageJSON referencing

Coordinate referencing metadata in CoverageJSON.

REQUIREMENT 19	
IDENTIFIER	`/req/data-query-response-format/referencing`
INCLUDED IN	Requirements class 4: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/data-query-response-format`
STATEMENT	CoverageJSON domain.referencing system id SHALL reflect the crs query parameter (default OGC:CRS84).
A	With no crs query param, domain.referencing.* system SHALL have type GeographicCRS and id OGC:CRS84.
B	With a crs query param, the system object with type GeographicCRS SHALL set id to the crs query param value.

A

ANNEX A (INFORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE (NORMATIVE)

ANNEX A (INFORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE (NORMATIVE)

CONFORMANCE CLASS A.1	
IDENTIFIER	`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/core`
REQUIREMENTS CLASS	Requirements class 1: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/core`
CONFORMANCE TESTS	Abstract test A.1-1: `/conf/core/openapi` Abstract test A.1-2: `/conf/core/collection_identifier` Abstract test A.1-3: `/conf/core/collection_title` Abstract test A.1-4: `/conf/core/collection_license` Abstract test A.1-5: `/conf/core/collection_spatial_extent` Abstract test A.1-6: `/conf/core/collection_temporal_extent` Abstract test A.1-7: `/conf/core/collection_parameter_names` Abstract test A.1-8: `/conf/core/collection_radius_data_query` Abstract test A.1-9: `/conf/core/locations_query_response_format` Abstract test A.1-10: `/conf/core/error_handling`

Example 1

label	/conf/core/openapi
subject	/req/core/openapi
test-purpose	Validate that a MetOcean EDR Profile provides an API definition using an OpenAPI document

Construct a path for a landing page.

Issue an HTTP GET request on that path.

Check that the value of the returned HTTP status header is 200.

In the `links` array in the landing page response, find the object with `rel` set to `service-desc` and `type` matching one of the following strings:

- `application/openapi+json`
- `application/openapi+yaml`

- `application/vnd.oai.openapi+json`
- `application/vnd.oai.openapi+yaml`

Optionally, the string may be suffixed with a version number like `;version=3.1`. In that case, check that the version is >= 3.1.

Issue an HTTP GET request using the value of `href` for that object.

Check that the value of the returned HTTP status header is 200 and check that the response body is a valid OpenAPI document of version 3.1 or higher.

In the `links` array in the landing page response, find the object with `rel` set to `service-doc`.

Issue an HTTP GET request using the value of `href` for that object.

Check that the value of the returned HTTP status header is 200 and that the HTTP response header `Content-Type` contains `text/html`.

Example 2

label	/conf/core/collection_identifier
subject	/req/core/collection_identifier
test-purpose	Validate that a MetOcean EDR Profile API provides a valid collection identifier

This requirement is not applicable to ATS testing.

Example 3

label	/conf/core/collection_title
subject	/req/core/collection_title
test-purpose	Validate that a MetOcean EDR Profile provides a valid collection title

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the `collections` array, check that the `title` property is present for all array elements.

For each `title` value, check that the value is less than or equal to 50 characters.

Example 4

label	/conf/core/collection_license
subject	/req/core/collection_license
test-purpose	Validate that a MetOcean EDR Profile API provides a collection links array with a license link.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the `links` array, check that there exists one `link` object with `rel` set to `license`.

Issue a HTTP GET request to the url in `href` for `link` with rel `license`.

Check that the value of the returned HTTP status header is 200 and the HTTP response header `Content-Type` is `text/html`.

Example 5

label	/conf/core/collection_spatial_extent
subject	/req/core/collection_spatial_extent
test-purpose	Validate that a MetOcean EDR Profile API provides spatial extent in the correct crs.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the response document, for each object in the `collections` array, check that the `extent.spatial.crs` property is set to `OGC:CRS84`.

In the `collections` array, check that the `crs` property is set to `OGC:CRS84` for each object.

For each object in `collections` array, for each object in the `data_queries` array, if `crs_details` is specified, check that one of the objects in the `crs_details` array has a `crs` property set to `OGC:CRS84`.

Example 6

label	/conf/core/collection_temporal_extent
subject	/req/core/collection_temporal_extent
test-purpose	Validate that a MetOcean EDR Profile API provides temporal extent in the correct trs.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the response document, for each object in the `collections` array, check that the `extent.temporal.trs` property is set to `Gregorian`.

Example 7

label	/conf/core/collection_parameter_names
subject	/req/core/collection_parameter_names
test-purpose	Validate that a MetOcean EDR Profile API provides a valid collection parameter_names

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the response document, for each object in the `collections` array, check that each object in the `parameter_names` array has the properties `label`, `unit` and `description`.

The value of a `label` property in a `parameter` object is less than or equal to 50 characters.

The value of `unit.symbol.type` in a `parameter` object is a url on the form `https://qudt.org/vocab/unit/{unit}`.

The value of `unit.symbol.value` in a `parameter` object is specified.

Example 8

label	/conf/core/collection_radius_data_query
subject	/req/core/collection_radius_data_query
test-purpose	Validate that a MetOcean EDR Profile API with a radius data query in a collection has correct metadata.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the response document, for each object in the `collections` array, if the `data_queries` property contains `radius`, check that the `link.variables` object for `radius` has a `within_units` array that contains `m`.

Example 9

label	/conf/core/locations_query_response_format
subject	/req/core/locations_query_response_format
test-purpose	Validate that a MetOcean EDR Profile API has a correctly formated response to a locations query.

For every collection and instance of a collection, check if `locations` exists as a key in the `data_queries` object. If so, get the `href` value from the link object with `"rel":"self"` and append `/locations` to the value and issue an HTTP GET request.

Check that the value of the returned HTTP status header is 200.

Check that the response http header `Content-Type` has value `application/geo+json`.

In the response document, check that it has the property `"type": "FeatureCollection"`. For each object in the `features` array, check that the properties `id` and `properties.name` are present.

Example 10

label	/conf/core/error_handling
subject	/req/core/error_handling
test-purpose	Validate that a MetOcean EDR Profile API handles errors correctly.

Issue an HTTP GET request to an non-existent path `/collections/nonexistentcollection`.

Check that the value of the returned HTTP status code is `404` and the HTTP response header `Content-Type` is `application/problem+json`.

Check that the returned JSON document includes the properties `type` and `title`. Check that `title` is a string and that `type` is either an URI or the value `about:blank`.

CONFORMANCE CLASS A.2	
IDENTIFIER	`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/insitu-observations`
REQUIREMENTS CLASS	Requirements class 2: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/insitu-observations`
CONFORMANCE TESTS	Abstract test A.2-1: `/conf/insitu-observations/collection_data_queries` Abstract test A.2-2: `/conf/insitu-observations/collection_parameter_names` Abstract test A.2-3: `/conf/insitu-observations/collection_custom_dimensions` Abstract test A.2-4: `/conf/insitu-observations/coveragejson_parameters`

Example 11

label	/conf/insitu-observations/collection_data_queries
subject	/req/insitu-observations/collection_data_queries
test-purpose	Validate that a MetOcean EDR Insitu observations Profile API has implemented the mandatory data queries.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the `collections` array in the returned document, check that each collection has a `data_queries` array containing `area`, `locations` and `radius`.

Example 12

label	/conf/insitu-observations/collection_parameter_names
subject	/req/insitu-observations/collection_parameter_names
test-purpose	Validate that a MetOcean EDR Insitu observations Profile API has the required properties for parameter_names

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

For each object in the `collections` array and for each object in `parameter_names` check that it has the properties `metocean:standard_name` with the value of type string.

For each object in the `collections` array and for each object in `parameter_names` check that it has the property `metocean:level` with a value of type number.

For each object in the `collections` array and for each object in `parameter_names` check that is has the property `measurementType`.

Example 13

label	/conf/insitu-observations/collection_custom_dimensions
subject	/req/insitu-observations/collection_custom_dimensions
test-purpose	Validate that a MetOcean EDR Insitu observations Profile API has required custom dimensions.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

Check that the `extent.custom` array has an object with property `id` set to `standard_name` and property `reference` set to `https://vocab.nerc.ac.uk/standard_name/`.

Check that the `extent.custom` array has an object with property `id` set to `level` and property `reference` set to `Height of measurement above ground level in meters`.

Example 14

label	/conf/insitu-observations/coveragejson_parameters
subject	/req/insitu-observations/coveragejson_parameters
test-purpose	Validate that a MetOcean EDR Insitu observations Profile API has required metadata in CoverageJSON parameters.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

For each object in the `collections` array and for each object in `data_queries`, issue a GET request with no crs query parameter specified.

Check that each object in `parameters` array of the response CoverageJSON document contains a property `metocean:standard_name` with a string type.

Check that each `parameter` object in `parameters` in the response CoverageJSON document contains a property `metocean:level` with a number type.

Check that each `parameter` object in `parameters` in the response CoverageJSON document contains a property `metocean:measurementType` which has an object type. Check that this object has the property `method` of type string and property `duration` of type string.

CONFORMANCE CLASS A.3	
IDENTIFIER	`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/nwp`
REQUIREMENTS CLASS	Requirements class 3: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/nwp`
CONFORMANCE TESTS	Abstract test A.3-1: `/conf/nwp/collection_parameter_names` Abstract test A.3-2: `/conf/nwp/collection_granularity` Abstract test A.3-3: `/conf/nwp/collection_data_queries`

Example 15

label	/conf/nwp/collection_parameter_names
subject	/req/nwp/collection_parameter_names
test-purpose	Validate that a MetOcean EDR API has implemented the mandatory parameter names for NWP.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the `collections` array in the returned document, check that each collection has a `data_queries` array containing `position` and `cube`.

For each matching collection `parameter_names` object, check that all of the Annex D ECMWF short names are present via each parameter name key, `id` and `description` properties.

Example 16

label	/conf/nwp/collection_granularity
subject	/req/nwp/collection_granularity
test-purpose	Validate that a MetOcean EDR API has implemented the mandatory granularity for NWP.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the `collections` array in the returned document, check that each collection has a `data_queries` array containing `position` and `cube`.

For matching collections, issue an HTTP GET request to path `/collections/{collectionId}/instances`.

For matching instances, ensure that the instance identifier (`id`) is an RFC3339 datestamp.

Example 17

label	/conf/nwp/collection_data_queries

subject	/req/nwp/collection_data_queries
test-purpose	Validate that a MetOcean EDR API has implemented the mandatory data query behaviour for NWP.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the `collections` array in the returned document, check that each collection has a `data_queries` array containing `position` and `cube`.

For a matching collection, issue an HTTP GET request to path `/collections/{collectionId}/position?coords=POINT(0 0)`.

Check that the value of the returned HTTP status header is 200.

For the same matching collection, issue an HTTP GET request to path `/collections/{collectionId}/instances`.

Check that the value of the returned HTTP status header is 200.

In the CoverageJSON response, ensure that the first value of `domain.axes.t.values` is equal to the last instance (`id`) from the `/collections/{collectionId}/instances` response captured earlier in this test.

CONFORMANCE CLASS A.4	
IDENTIFIER	`http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/conf/data-query-response-format`
REQUIREMENTS CLASS	Requirements class 4: `http://www.opengis.net/spec/profile/metocean-ogcapi-environmental-data-retrieval/1.0/req/data-query-response-format`
CONFORMANCE TESTS	Abstract test A.4-1: `/conf/data-query-response-format/encoding` Abstract test A.4-2: `/conf/data-query-response-format/referencing`

Example 18

label	/conf/data-query-response-format/encoding
subject	/req/data-query-response-format/encoding
test-purpose	Validate that a MetOcean EDR API has the correct response format for data queries.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

In the returned document, for each object in `data_queries` for each collection in `collections` make a corresponding data query request and check that the returned response has status 200 and has an HTTP response header `Content-Type` with value `application/vnd.cov+json`.

Example 19

label	/conf/data-query-response-format/referencing
subject	/req/data-query-response-format/referencing
test-purpose	Validate that a MetOcean EDR Profile API CoverageJSON response document provides correct domain referencing values.

Issue an HTTP GET request to path `/collections`.

Check that the value of the returned HTTP status header is 200.

For each object in the `collections` array, for each object in `data_queries`, issue a GET request with no crs query parameter specified.

Check that the response is a CoverageJSON document where each object of `type Coverage` contains a `domain.referencing` array which includes an object where `system.type` is set to `GeographicCRS` and `system.id` set to `OGC:CRS84`.

For each object in the `collections` array, for each object in `data_queries`, issue a GET request for each `crs_details` object where `crs` query parameter set to the value in the `crs_details` object.

Check that the response is a CoverageJSON object where `domain.referencing` includes an object where `system.type` is set to `GeographicCRS` and `system.id` is equal to the crs query parameter.

B

ANNEX B (INFORMATIVE) SCHEMAS (NORMATIVE)

B ANNEX B (INFORMATIVE) SCHEMAS (NORMATIVE)

C

ANNEX C (INFORMATIVE) EXAMPLES (INFORMATIVE)

ANNEX C (INFORMATIVE) EXAMPLES (INFORMATIVE)

C.1. Examples

D

ANNEX D (INFORMATIVE) NWP GLOBAL DETERMINISTIC PARAMETER NAME MATRIX

ANNEX D (INFORMATIVE) NWP GLOBAL DETERMINISTIC PARAMETER NAME MATRIX

Table D.1

PARAMETER	ECMWF SHORT NAME
Geopotential Height	gh
Temperature	t
Wind zonal velocity (u)	u
Wind meridional velocity (v)	v
Relative Humidity	r
Mean Sea Level Pressure (MSLP)	msl
2-m Temperature	2t
2-m minimum temperature in the last 3 hours	mn2t3
2-m maximum temperature in the last 3 hours	mx2t3
2-m minimum temperature in the last 6 hours	mn2t6
2-m maximum temperature in the last 6 hours	mx2t6
2m dewpoint temperature	2d
10m u	10u
10m v	10v
10m wind gust	10fg
Total Precipitation	tp

PARAMETER	ECMWF SHORT NAME
Total solid precipitation	TODO fix
CAPE	cape
Total precipitable water	tcw
Total cloud cover	tcc

E

ANNEX E (INFORMATIVE) REVISION HISTORY

ANNEX E (INFORMATIVE) REVISION HISTORY

Table E.1

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2024-09-16	Template	Tom Kralidis	all	Initial template
2026-01-16	Draft	Tom Kralidis	all	Add requirements and conformance for NWP

BIBLIOGRAPHY

BIBLIOGRAPHY

[1] W3C: Data Catalog Vocabulary, W3C Recommendation, 16 January 2014, https://www.w3.org/TR/vocab-dcat

[2] W3C: Data on the Web Best Practices, W3C Recommendation, 31 January 2017, https://www.w3.org/TR/dwbp

[3] W3C/OGC: Spatial Data on the Web Best Practices, W3C Working Group Note, 28 September 2017, https://www.w3.org/TR/sdw-bp

[4] Linux Foundation: SPDX License List, https://spdx.org/licenses

[5] IANA: Link Relation Types, https://www.iana.org/assignments/link-relations/link-relations.xml